

Lab 16 - NetworkPolicy and Default-Deny Isolation

Field	Value
Lab ID	Lab 16
Day / Topic	Day 3 Security
Duration	50 minutes
Namespace	kcna-lab16
Mapping	LO4 A4 K6
Cluster	Single-node kind cluster, Kubernetes v1.30 or later

☐ Read this before you start - the honesty statement for this lab

>

kind's default CNI (kindnet) does not enforce NetworkPolicy.

>

The networking.k8s.io/v1 NetworkPolicy API is served by the API server on every conformant cluster, so every object in this lab will be accepted and stored. But a NetworkPolicy is enforced by the CNI plugin, and kindnet - the CNI kind installs by default - implements Pod networking without a policy engine.

>

Consequence: after you apply default-deny-all, the reporting Pod will **still** be able to reach payments-api on a stock kind cluster. ****That is not a mistake in this lab, and it is not a mistake in your YAML.**** It is the CNI.

>

This lab therefore teaches you to verify a policy by reasoning about the object - selectors, ports, direction, namespace scope - and gives you an explicit enforcement probe whose result you record honestly either way. Appendix A shows how to build a kind cluster with Calico if you want real enforcement.

>

Nothing in this README shows a connection being blocked on a cluster where it was not blocked.

1. Objective

By the end of this lab you will be able to:

1. Read every field of a networking.k8s.io/v1 NetworkPolicy: podSelector,

policyTypes, ingress, egress, namespaceSelector, ports.

2. Explain the additive, deny-by-default-on-selection model - that NetworkPolicy has no deny verb, and that isolation is a *consequence* of being selected.
3. Build the default-deny then selective-allow pattern, including the DNS egress rule that default-deny otherwise breaks.
4. State which port number a NetworkPolicy matches (the Pod port, not the Service port) and why.
5. Determine whether your cluster's CNI enforces NetworkPolicy at all, and interpret the result correctly instead of assuming.
6. Diagnose a policy that is stored but ineffective because a selector or port does not match anything.

2. Prerequisites

- Labs 11 and 14 completed. You need Services/EndpointSlices, and the Lab 14 point that a namespace is not a network boundary.
- A running single-node kind cluster.
- Working directory:

```
cd courseware/labs/lab-16-networkpolicy-isolation
pwd
```

Expected output:

```
/.../courseware/labs/lab-16-networkpolicy-isolation
```

- Confirm the API is served (it always is - this proves nothing about enforcement):

```
kubectl api-resources --api-group=networking.k8s.io | grep networkpolicies
```

Expected output:

```
networkpolicies  netpol  networking.k8s.io/v1  true  NetworkPolicy
```

- Identify your CNI now, because it determines what you will observe:

```
kubectl get daemonsets -A
```

Expected output on a stock kind cluster:

NAMESPACE	NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR
	AGE						
kube-system	kindnet	1	1	1	1	1	<none>
	58m						
kube-system	kube-proxy	1	1	1	1	1	kubernetes.io/
os=linux	58m						

On the documented stock kind setup, kindnet provides no NetworkPolicy enforcement. Inventory all namespaces and confirm the actual behaviour in Step 5; names alone do not establish every possible CNI or external policy configuration.

3. Scenario

Meridian Freight's auditors have flagged a finding: any Pod in the cluster can open a TCP connection to any other Pod. The payments-api holds settlement data and should only be reachable from the shipper portal front end. The finance reporting job runs in the same namespace and currently has full access to it.

You will implement the standard remediation - default-deny, then allow exactly one flow - and then do the thing most tutorials skip: *check whether the cluster you are running on can actually enforce what you wrote.*

4. Step-by-step procedure

Step 1 - Namespace, backend and two clients

```
kubectl apply -f manifests/00-namespace.yaml
kubectl apply -f manifests/10-configmap-payments.yaml
kubectl apply -f manifests/20-deployment-payments-api.yaml
kubectl apply -f manifests/30-pods-clients.yaml
```

Expected output:

```
namespace/kcna-lab16 created
configmap/payments-content created
configmap/payments-nginx created
deployment.apps/payments-api created
service/payments-api created
pod/frontend created
pod/reporting created
```

```
kubectl wait --for=condition=Available deployment/payments-api -n kcna-lab16 --timeout=120s
kubectl wait --for=condition=Ready pod/frontend pod/reporting -n kcna-lab16 --timeout=90s
```

Expected output:

```
deployment.apps/payments-api condition met
pod/frontend condition met
pod/reporting condition met
```

Look carefully at the labels - they are the entire mechanism of this lab:

```
kubectl get pods -n kcna-lab16 --show-labels
```

Expected output (Pod-name suffix will differ):

NAME	READY	STATUS	RESTARTS	AGE	LABELS
frontend	1/1	Running	0	32s	app=shipper-portal, role=frontend
payments-api-5f6d84b7c9-gq2xn	1/1	Running	0	32s	app.kubernetes.io/part-of=meridian-freight,app=payments-api,pod-template-hash=5f6d84b7c9,tier=backend
reporting	1/1	Running	0	32s	app=finance-reporting, role=reporting

frontend and reporting are the same image with the same command. The only difference between them is role=frontend versus role=reporting.

Step 2 - Establish the baseline: everything can reach everything

```
kubectl exec -n kcna-lab16 frontend -- wget -q0- --timeout=5 http://payments-api/ | grep sensitivity
```

Expected output:

```
<p>sensitivity: RESTRICTED - only role=frontend should reach this</p>
```

```
kubectl exec -n kcna-lab16 reporting -- wget -q0- --timeout=5 http://payments-api/ | grep sensitivity
```

Expected output:

```
<p>sensitivity: RESTRICTED - only role=frontend should reach this</p>
```

That second result is the audit finding. With no NetworkPolicy anywhere, the Kubernetes network model is deliberately flat: every Pod can reach every Pod, across namespaces, without NAT.

Confirm there is genuinely no policy yet:

```
kubectl get networkpolicies -n kcna-lab16
```

Expected output:

```
No resources found in kcna-lab16 namespace.
```

Step 3 - Apply default-deny

```
kubectl apply -f manifests/40-networkpolicy-default-deny.yaml
```

Expected output:

```
networkpolicy.networking.k8s.io/default-deny-all created
```

```
kubectl describe networkpolicy default-deny-all -n kcna-lab16
```

Expected output:

```

Name:      default-deny-all
Namespace:  kcna-lab16
Created on: 2026-09-05 11:04:22 +0800 +08
Labels:    app.kubernetes.io/part-of=meridian-freight
Annotations: <none>
Spec:
  PodSelector:      <none> (Allowing the specific traffic to all pods in this namespace)
  Allowing ingress traffic:
    <none> (Selected pods are isolated for ingress connectivity)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress connectivity)
  Policy Types: Ingress, Egress

```

Read those three parenthesised phrases - they are kubectl's own summary of the model:

Line	Meaning
PodSelector: <none> (... to all pods in this namespace)	podSelector: {} selects every Pod. <none> here means "no *constraint*", not "no Pods"
Allowing ingress traffic: <none> (Selected pods are isolated...)	Zero allow rules. Being *selected* is what causes isolation
Policy Types: Ingress, Egress	Both directions are now governed for those Pods

The model, stated exactly:

- NetworkPolicy has no deny rule. You cannot write one.
- A Pod is *non-isolated* for a direction until some policy selects it for that

direction. Then it becomes deny-by-default in that direction.

- What gets through is the union of the allow rules of every policy that selects

the Pod. Policies are purely additive and unordered.

- Consequently, you cannot "block one client". You deny everything and re-allow the

flows you want.

Step 4 - Restore DNS, then allow exactly one flow

A default-deny egress policy also blocks DNS, because a DNS query is just UDP/53 to CoreDNS in kube-system. On an enforcing CNI, forgetting this rule turns every `wget http://payments-api/` into bad address 'payments-api' - which looks like a DNS outage and sends people to debug CoreDNS for an hour.

```

kubectl apply -f manifests/50-networkpolicy-allow-dns.yaml
kubectl apply -f manifests/60-networkpolicy-allow-frontend.yaml

```

Expected output:

```

networkpolicy.networking.k8s.io/allow-dns-egress created
networkpolicy.networking.k8s.io/allow-frontend-to-payments created
networkpolicy.networking.k8s.io/allow-frontend-egress-to-payments created

```

```

kubectl get networkpolicies -n kcna-lab16

```

Expected output:

NAME	POD-SELECTOR	AGE
allow-dns-egress	<none>	12s
allow-frontend-egress-to-payments	role=frontend	12s
allow-frontend-to-payments	app=payments-api	12s
default-deny-all	<none>	3m

```
kubectl describe networkpolicy allow-frontend-to-payments -n kcna-lab16
```

Expected output:

```
Name:      allow-frontend-to-payments
Namespace: kcna-lab16
Labels:    app.kubernetes.io/part-of=meridian-freight
Spec:
  PodSelector:  app=payments-api
  Allowing ingress traffic:
    To Port: 8080/TCP
    From:
      PodSelector: role=frontend
  Not affecting egress traffic
  Policy Types: Ingress
```

Four things to internalise from this one object:

1. PodSelector: app=payments-api - the policy is attached to the server, not to the client. NetworkPolicy always selects the Pods it *protects*.
2. To Port: 8080/TCP - this is the Pod port (containerPort: 8080), not the Service port 80. Policy is evaluated after kube-proxy has already DNATed the destination to a Pod IP and Pod port. Writing port: 80 here is one of the most common silent mistakes, and section 6 makes you do it.
3. From: PodSelector: role=frontend - a bare podSelector means "in this namespace". To allow another namespace you need a namespaceSelector, as allow-dns-egress does for kube-system.
4. Not affecting egress traffic - policyTypes: [Ingress] here. The client's outbound permission is a separate decision, which is why allow-frontend-egress-to-payments exists.

Look at the namespace-crossing rule too:

```
kubectl get networkpolicy allow-dns-egress -n kcna-lab16 -o jsonpath='{.spec.egress[0].to}' ; echo
```

Expected output:

```
[{"namespaceSelector":{"matchLabels":{"kubernetes.io/metadata.name":"kube-system"}},
"podSelector":{"matchLabels":{"k8s-app":"kube-dns"}}}]
```

The single subtlest rule in the whole API. namespaceSelector and podSelector in the same list item are ANDed - "a Pod labelled k8s-app=kube-dns *in* kube-system". Split them into two list items and they become ORed - "anything in kube-system, or any k8s-app=kube-dns Pod anywhere". A misplaced hyphen changes the meaning completely and nothing warns you.

Step 5 - Determine whether this cluster enforces any of it

Now the honest part. Re-run the two probes from Step 2 and record what actually happens.

```
echo "--- frontend -> payments-api (policy INTENDS: allow) ---"
kubectl exec -n kcna-lab16 frontend -- wget -q0- --timeout=5 http://payments-api/ >/dev/null
2>&1 \
  && echo "REACHABLE" || echo "BLOCKED"
echo "--- reporting -> payments-api (policy INTENDS: deny) ---"
kubectl exec -n kcna-lab16 reporting -- wget -q0- --timeout=5 http://payments-api/ >/dev/null
2>&1 \
  && echo "REACHABLE" || echo "BLOCKED"
```

Expected output on a stock kind cluster (kindnet):

```
--- frontend -> payments-api (policy INTENDS: allow) ---
REACHABLE
--- reporting -> payments-api (policy INTENDS: deny) ---
REACHABLE
```

Expected output on a cluster with an enforcing CNI (Calico, Cilium, Antrea, and the managed CNIs on EKS/GKE/AKS):

```
--- frontend -> payments-api (policy INTENDS: allow) ---
REACHABLE
--- reporting -> payments-api (policy INTENDS: deny) ---
BLOCKED
```

If you saw REACHABLE for reporting, your policy is not wrong - your CNI is not enforcing it. Confirm the cause rather than guessing:

```
kubectl get daemonsets -A -o custom-columns='NAME:.metadata.name, IMAGE:.spec.template.spec.
containers[0].image'
```

Expected output on stock kind:

NAME	IMAGE
kindnet	docker.io/kindest/kindnetd:v20240813-c6f155d6
kube-proxy	registry.k8s.io/kube-proxy:v1.31.0

There is no policy engine in that list. The reasoning chain, which you should be able to state in an interview:

1. The NetworkPolicy API is part of Kubernetes; enforcement is not.
2. Enforcement is delegated to the CNI plugin.
3. kindnet provides Pod networking (IPAM, routes, port mapping) but implements **no NetworkPolicy controller**.
4. Therefore, on stock kind, a NetworkPolicy is an inert, correctly stored object.
5. Calico, Cilium and Antrea can enforce it. Managed-cloud enforcement depends on the selected CNI and enabled policy features or add-ons; verify that setup explicitly.

There is no event, condition, status field or warning anywhere that tells you this. `kubectl get netpol` looks identical on both kinds of cluster. The only ways to know are to inspect the CNI, or to probe - which is exactly what you just did.

Step 6 - Verify the policy by reasoning, not by observation

Since the training cluster cannot demonstrate enforcement, you verify the *design*. This is a genuinely useful professional skill: policy review before rollout.

The intended traffic is recorded in data/traffic-matrix.csv:

```
cat data/traffic-matrix.csv
```

Expected output:

```
source,source_selector,destination,dest_selector,port,protocol,intended,expressed_by
frontend,role=frontend,payments-api,app=payments-api,8080,TCP,allow,allow-frontend-to-payments
reporting,role=reporting,payments-api,app=payments-api,8080,TCP,deny,no allow rule matches -
default-deny-all applies
frontend,role=frontend,kube-dns,k8s-app=kube-dns,53,UDP,allow,allow-dns-egress
reporting,role=reporting,kube-dns,k8s-app=kube-dns,53,UDP,allow,allow-dns-egress
frontend,role=frontend,internet,none,443,TCP,deny,default-deny-all egress
payments-api,app=payments-api,frontend,role=frontend,8080,TCP,deny,frontend has no ingress
allow rule
```

Trace row 2 by hand - the flow the auditors care about, reporting → payments-api:8080:

Question	Command	Answer
Is the destination selected by any Ingress policy?	kubectl get netpol -n kcna-lab16 -o jsonpath=...	Yes - default-deny-all ({}) and allow-frontend-to-payments (app=payments-api)
So is it isolated for ingress?	-	Yes. Selected ☐ deny-by-default
Does any allow rule admit the source?	allow-frontend-to-payments allows role=frontend	reporting is role=reporting ☐ no
Verdict	-	DENY - which matches the matrix

Verify each link of that chain against the live cluster:

```
kubectl get pods -n kcna-lab16 -l app=payments-api -o name
```

Expected output - the destination selector does match a real Pod:

```
pod/payments-api-5f6d84b7c9-gq2xn
```

```
kubectl get pods -n kcna-lab16 -l role=frontend -o name
kubectl get pods -n kcna-lab16 -l role=reporting -o name
```

Expected output - the allow-list selector matches exactly one Pod, and it is not reporting:

```
pod/frontend
pod/reporting
```

```
kubectl get networkpolicy allow-frontend-to-payments -n kcna-lab16 \
-o jsonpath='{.spec.ingress[0].ports[0].port}/{ "/" }{.spec.ingress[0].ports[0].protocol}{ "\n" }'
kubectl get pods -n kcna-lab16 -l app=payments-api \
-o jsonpath='{.items[0].spec.containers[0].ports[0].containerPort}{ "\n" }'
```

Expected output - these two numbers must be equal, and they are:

```
8080/TCP
8080
```

That three-way check - *destination selector matches a Pod*, *source selector matches the intended client and no one else*, *policy port equals the container port* - catches the overwhelming majority of real NetworkPolicy defects, and it works on any cluster, enforcing or not.

5. Verification

```
bash verification/checks.sh
```

Expected output on a stock kind cluster:

```
== Lab 16 verification: NetworkPolicy and default-deny isolation ==
PASS namespace kcna-lab16 exists
PASS ConfigMap payments-content matches data/payments-index.html byte-for-byte
PASS deployment payments-api is Available and Service payments-api has endpoints
PASS frontend and reporting run the same image and differ by the role label
PASS default-deny-all selects ALL pods ({} ) for both Ingress and Egress
PASS default-deny-all declares no allow rules (isolation by selection)
PASS allow-dns-egress permits UDP/53 and TCP/53 to kube-system
PASS allow-dns-egress ANDs namespaceSelector with podSelector in one peer
PASS allow-frontend-to-payments protects app=payments-api on ingress
PASS allow-frontend-to-payments admits role=frontend only
PASS allow-frontend-to-payments port 8080 equals the payments-api containerPort
PASS allow-frontend-egress-to-payments covers the client side of the same flow
PASS matrix allow: frontend -> payments-api:8080 is expressed by a policy
PASS matrix deny: reporting -> payments-api:8080 is expressed by NO policy
PASS matrix deny: payments-api -> frontend:8080 is expressed by NO policy
NOTE -----
NOTE CNI DaemonSet(s) detected: kindnet
NOTE kindnet does NOT implement NetworkPolicy enforcement.
NOTE Enforcement probe: reporting -> payments-api was REACHABLE.
NOTE That is the EXPECTED result on stock kind and is NOT a policy defect.
NOTE Appendix A shows how to build a kind cluster with Calico for real
NOTE enforcement. The policy OBJECTS above are verified as correct.
NOTE -----
PASS RECORDED: enforcement state of this cluster determined and reported honestly
-- 16 passed, 0 failed --
```

On a cluster with an enforcing CNI the NOTE block changes and two extra assertions run - see verification/expected-output.md.

6. Failure injection - a policy that is stored and does nothing

6.1 Break it

```
kubectl apply -f manifests/70-networkpolicy-broken-selector.yaml
```

Expected output - accepted, with no complaint whatsoever:

```
networkpolicy.networking.k8s.io/allow-frontend-to-payments-broken created
```

```
kubectl get networkpolicies -n kcna-lab16
```

Expected output - it sits in the list looking exactly as legitimate as the others:

NAME	POD-SELECTOR	AGE
allow-dns-egress	<none>	6m
allow-frontend-egress-to-payments	role=frontend	6m
allow-frontend-to-payments	app=payments-api	6m
allow-frontend-to-payments-broken	app=payments	8s
default-deny-all	<none>	9m

6.2 Diagnose - three independent defects

```
kubectl describe networkpolicy allow-frontend-to-payments-broken -n kcna-lab16
```

Expected output:

```
Name:          allow-frontend-to-payments-broken
Namespace:     kcna-lab16
Labels:        kcna.tertiaryinfotech.com/intent=failure-injection
Spec:
  PodSelector:  app=payments
  Allowing ingress traffic:
    To Port: 80/TCP
    From:
      PodSelector: role=frontends
  Not affecting egress traffic
  Policy Types: Ingress
```

Run each selector as a live query - the only reliable way to check a NetworkPolicy:

Defect 1 - the policy protects nothing.

```
kubectl get pods -n kcna-lab16 -l app=payments
```

Expected output:

```
No resources found in kcna-lab16 namespace.
```

The label is app=payments-api. app=payments matches zero Pods, so this policy attaches to nothing.

Defect 2 - the port is the Service port, not the Pod port.

```
kubectl get networkpolicy allow-frontend-to-payments-broken -n kcna-lab16 \
-o jsonpath='netpol port: {.spec.ingress[0].ports[0].port}{"\n"}'
kubectl get service payments-api -n kcna-lab16 \
-o jsonpath='service port: {.spec.ports[0].port} -> pod port: {.spec.ports[0].targetPort}{"\n"}'
```

Expected output:

```
netpol port: 80
service port: 80 -> pod port: http
```

The policy says 80; traffic arrives at the Pod on 8080, because kube-proxy already translated it. NetworkPolicy sits below the Service abstraction.

Defect 3 - the source selector matches nothing.

```
kubectl get pods -n kcna-lab16 -l role=frontends
```

Expected output:

```
No resources found in kcna-lab16 namespace.
```

role=frontends $\neq$ role=frontend.

Diagnosis. Three defects, three silent failures, zero errors, zero events, zero status. And note the specific danger: because NetworkPolicy is additive, a broken **allow** policy never opens a hole - it simply fails to open the one you intended, so your legitimate traffic breaks while you stare at a policy that "looks right". The mirror-image danger is worse: a broken default-deny (for example, a podSelector that matches nothing) leaves you believing you are isolated when you are not.

Confirm that last point:

```
kubectl get networkpolicy default-deny-all -n kcna-lab16 -o jsonpath='{.spec.podSelector}' ;
echo " <- {} means ALL pods"
kubectl get pods -n kcna-lab16 --no-headers | wc -l
```

Expected output:

```
{ } <- { } means ALL pods
3
```

An empty podSelector covers all 3 Pods. If it had said matchLabels: {app: payments} it would cover 0, and your "default deny" would deny nothing at all.

6.3 Fix and re-verify

```
kubectl delete networkpolicy allow-frontend-to-payments-broken -n kcna-lab16
```

Expected output:

```
networkpolicy.networking.k8s.io "allow-frontend-to-payments-broken" deleted
```

Re-run the three-way reasoning check on the **correct** policy:

```
kubectl get pods -n kcna-lab16 -l "$(kubectl get netpol allow-frontend-to-payments -n kcna-lab16 -o jsonpath='app={.spec.podSelector.matchLabels.app}')" -o name
kubectl get pods -n kcna-lab16 -l "$(kubectl get netpol allow-frontend-to-payments -n kcna-lab16 -o jsonpath='role={.spec.ingress[0].from[0].podSelector.matchLabels.role}')" -o name
```

Expected output:

```
pod/payments-api-5f6d84b7c9-gq2xn
pod/frontend
```

Both selectors resolve to exactly the intended Pod. The policy is correct - whether or not this cluster will act on it.

7. Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix	
Policy applied but traffic still flows	The CNI does not enforce NetworkPolicy (kindnet, and some minimal CNIs)	kubectl get ds -A and the cluster configuration show stock kindnet without a policy engine	Expected on stock kind. Use Calico/Cilium (Appendix A) if you need enforcement	
Policy applied and *still* flows on Calico/Cilium	podSelector matches no Pod	kubectl get pods -n <ns> -l <selector> → No resources found	Fix the label; re-run the selector as a query	
Legitimate traffic blocked after adding an allow rule	Policy port is the Service port, not the container port	Compare netpol ...ports[0].port with pod ...containers[0].ports[0].containerPort	Use the Pod port	
Everything resolves as bad address after default-deny	Egress deny also blocks DNS	`kubectl get netpol -n <ns> -o yaml \	grep -A5 egress`	Add the allow-dns-egress policy (UDP and TCP 53 to kube-system)
Cross-namespace traffic blocked though a podSelector allows it	A bare podSelector only means "this namespace"	kubectl get netpol <n> -o jsonpath='{.spec.ingress[0].from}'	Add a namespaceSelector	
A rule allows far more than intended	namespaceSelector and podSelector were put in separate list items (OR) instead of one item (AND)	Count the - entries under from/to:	Merge them into a single list item	
Client allowed by the server's ingress rule but still cannot connect	Default-deny egress also governs the client	kubectl get netpol -n <ns> -o custom-columns=NAME:.metadata.name,TYPE:.spec.policyTypes	Add an egress allow policy for the client Pod	
kubectl describe netpol shows PodSelector: <none> and you panic	<none> means "no constraint" - i.e. all Pods	Read the parenthesised text kubectl prints beside it	Nothing to fix	
Policy has no effect on traffic from outside the cluster	Node-local and some LB traffic can bypass Pod-level policy depending on CNI and externalTrafficPolicy	Test from a Pod, not from the host	Test in-cluster; treat host-network sources separately	

8. Cleanup

Delete only this lab's namespace. NetworkPolicies are namespaced, so they go with it.

```
kubectl delete namespace kcna-lab16
```

Expected output:

```
namespace "kcna-lab16" deleted
```

Confirm:

```
kubectl get namespace kcna-lab16
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab16" not found
```

This lab creates no cluster-scoped objects. Never run

`kubectl delete netpol --all without -n kcna-lab16`; in a production namespace that

removes every isolation guarantee at once.

>

If you built the Appendix A cluster, remove it separately:

```
kind delete cluster --name kcna-netpol.
```

9. What you learned

- The NetworkPolicy API is universal; enforcement is not. The API server accepts and stores policies on every cluster. The CNI plugin enforces them - and kindnet, kind's default CNI, does not. There is no status, event or warning that reveals this; you must inspect the CNI or probe.
- The default Kubernetes network model is flat: with no policy, every Pod can reach every Pod in every namespace. A Namespace is not a network boundary (Lab 14).
- NetworkPolicy has no deny rule. A Pod becomes deny-by-default in a direction the moment *any* policy selects it for that direction; allowed traffic is the union of all matching allow rules.
- `podSelector: {}` means all Pods in the namespace - and a `podSelector` that matches nothing produces a policy that silently does nothing.
- The ports in a policy are Pod ports, not Service ports. Policy is evaluated after kube-proxy's translation.
- Default-deny egress breaks DNS. Always pair it with an allow rule for UDP *and* TCP 53 to kube-system.
- `namespaceSelector + podSelector` in one peer item is AND; in two items it is OR. One hyphen changes the security posture.
- Ingress on the server and egress on the client are separate decisions; a flow needs both.

- Verify a policy by running its selectors as live label queries and by comparing its

port with the container port. That review works on any cluster, enforcing or not.

10. Further reading

- Network Policies - <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- Declare Network Policy (walkthrough) - <https://kubernetes.io/docs/tasks/administer-cluster/declare-network-policy/>
- The Kubernetes network model - <https://kubernetes.io/docs/concepts/services-networking/#the-kubernetes-network-model>
- Network Plugins (CNI) - <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>
- kind - Configuration, disableDefaultCNI - <https://kind.sigs.k8s.io/docs/user/configuration/#disable-default-cni>
- Security concepts / cluster hardening - <https://kubernetes.io/docs/concepts/security/>

Appendix A - a complete policy-enforcing kind cluster

This optional path was tested with kind v0.33.0, Kubernetes v1.36.4 and Calico v3.32.2. Calico's published tested range is Kubernetes 1.34-1.36; do not replace the pinned node image with kind's default version without checking compatibility. The exact operator manifests used in verification are supplied in `data/calico-v3.32.2/`, with provenance, checksums and the upstream licence. Internet access is required to pull container images. Allow several minutes for first-time pulls; all six Calico status resources became Available in the test.

Run these commands from this lab folder, using Docker, kind and kubectl on macOS/Linux or a Linux shell such as WSL2 on Windows. This creates a separate single-node training cluster. The explicit kubeconfig leaves your existing kubectl context unchanged. If `kcna-netpol` already exists, inspect it before creating or reusing anything; do not apply this bootstrap to an unrelated cluster.

A1 - create the isolated cluster

```
KCNA_POLICY_DIR="$(mktemp -d)"
KCNA_POLICY_CFG="$KCNA_POLICY_DIR/kubeconfig"
kind create cluster --name kcna-netpol \
  --config data/kind-cluster-calico.yaml \
  --image kindest/node:v1.36.4@sha256:
099e049362a1526b2db71494e1947aae99bd16290d7c895f2b7ea312e3cbfaed \
  --kubeconfig "$KCNA_POLICY_CFG"
chmod 600 "$KCNA_POLICY_CFG"
kubectl --kubeconfig "$KCNA_POLICY_CFG" get nodes
```

NotReady is expected at this stage: the supplied kind configuration disables kindnet, and Calico has not been installed. Keep this terminal open so the two variables remain available. Treat the kubeconfig as a credential; do not copy it into the lab repository or evidence submission.

A2 - install the supplied, pinned Calico resources

```
kubectl --kubeconfig "$KCNA_POLICY_CFG" create -f data/calico-v3.32.2/v1_crd_projectcalico_org.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" create -f data/calico-v3.32.2/tigera-operator.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" rollout status deployment/tigera-operator -n tigera-operator --timeout=300s
kubectl --kubeconfig "$KCNA_POLICY_CFG" create -f data/calico-v3.32.2/custom-resources.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=create tigerastatus/calico --timeout=300s
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Available tigerastatus/calico --timeout=600s
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Ready node --all --timeout=300s
kubectl --kubeconfig "$KCNA_POLICY_CFG" get tigerastatus
kubectl --kubeconfig "$KCNA_POLICY_CFG" get daemonsets -A
```

Expect calico-node in calico-system, not necessarily kube-system, and the node to be Ready. The optional API server, Goldmane and Whisker components may finish after core networking. Before using those components, check their own Available status. If a wait times out, inspect get pods -A, get events -A and the affected Pod's describe output for pull or readiness failures; do not assume an accepted Installation object proves working networking.

A3 - establish a working baseline before policy

```
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/00-namespace.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/10-configmap-payments.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/20-deployment-payments-api.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/30-pods-clients.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Available deployment/payments-api -n kcna-lab16 --timeout=180s
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Ready pod/frontend pod/reporting -n kcna-lab16 --timeout=180s
kubectl --kubeconfig "$KCNA_POLICY_CFG" exec -n kcna-lab16 frontend -- wget -qO- --timeout=5 http://payments-api/
kubectl --kubeconfig "$KCNA_POLICY_CFG" exec -n kcna-lab16 reporting -- wget -qO- --timeout=5 http://payments-api/
```

Both clients must return the payments mock HTML. If either fails, repair the backend, Service, DNS or CNI before adding policy. A pre-existing connection failure would make a later deny result inconclusive.

A4 - apply isolation and prove the intended exception

```
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/40-networkpolicy-default-deny.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/50-networkpolicy-allow-dns.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/60-networkpolicy-allow-frontend.yaml
KUBECONFIG="$KCNA_POLICY_CFG" bash verification/checks.sh
```

The tested result was 18 passed, 0 failed. In particular:

- frontend -> payments-api remains REACHABLE.
- reporting -> payments-api is BLOCKED.
- DNS permits both UDP and TCP port 53; the allow flow covers both ingress and

the frontend's egress, because the default-deny policy isolates both directions.

This environment assignment applies only to the verification command. Other kubectl commands continue to use your original configuration. Save the baseline, policy inventory and post-policy outcomes as evidence; submit no kubeconfig.

A5 - finish the optional environment

The existing training context was never changed, so no hard-coded context switch is needed. Stop this optional cluster's container when finished to release CPU and memory while retaining its state:

```
docker stop kcna-netpol-control-plane
```

To resume the same cluster, start that container and use the retained kubeconfig; confirm readiness before continuing. Keep its private temporary directory only for as long as you need this training environment.

Sources: [Calico requirements](#), [Calico quickstart](#), [kind v0.33.0 node image release](#).